Taking storage-related (good) decisions

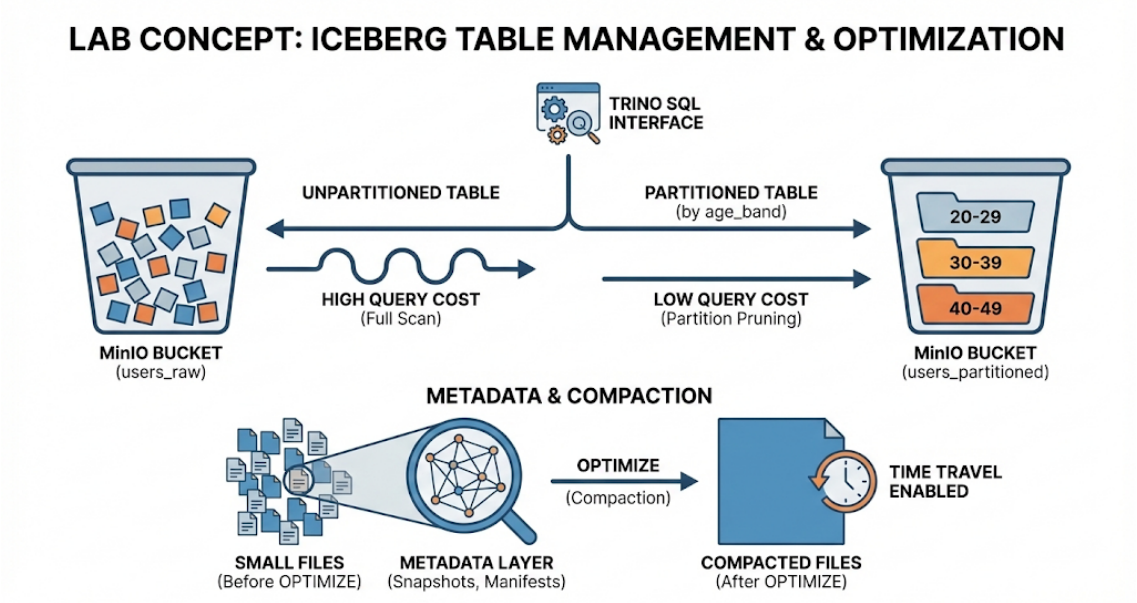
The focus of this practical assignment is:

- Use an object storage layout (MinIO bucket + objects)
- Use Parquet as the file format (what actually gets written)
- Argue about partitioning choices and partition pruning
- Deal with small files problem + compaction
- Use Iceberg metadata (snapshots, partitions, files, properties)
- Employ a "Catalog mindset" (tables are addressed by name, not by folders)

We'll use Trino (CLI or UI) to create / interact with data, through an Iceberg catalog configured to write to MinIO.

We'll check MinIO Console (UI) to browse the buckets / objects.

Please check the documentation for Trino's Iceberg connector for full reference of the connectors capabilities: https://trino.io/docs/current/connector/iceberg.html#

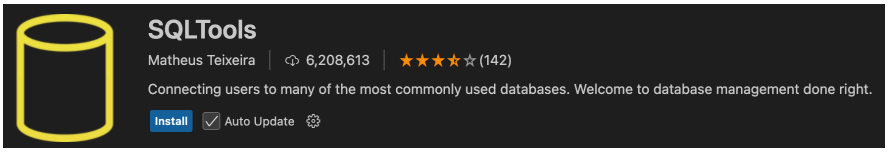


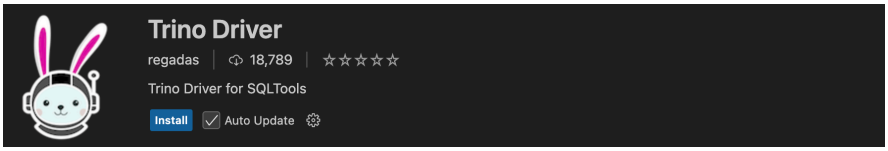
Scenario

You're building a `Users` table as an Iceberg table stored on MinIO. Your job is to pick, evaluate and argue about storage choices: partitioning, file sizing, and maintenance. These choices will have an impact on the way data is organized, and ultimately, on the performance of queries and other consumers of this data.

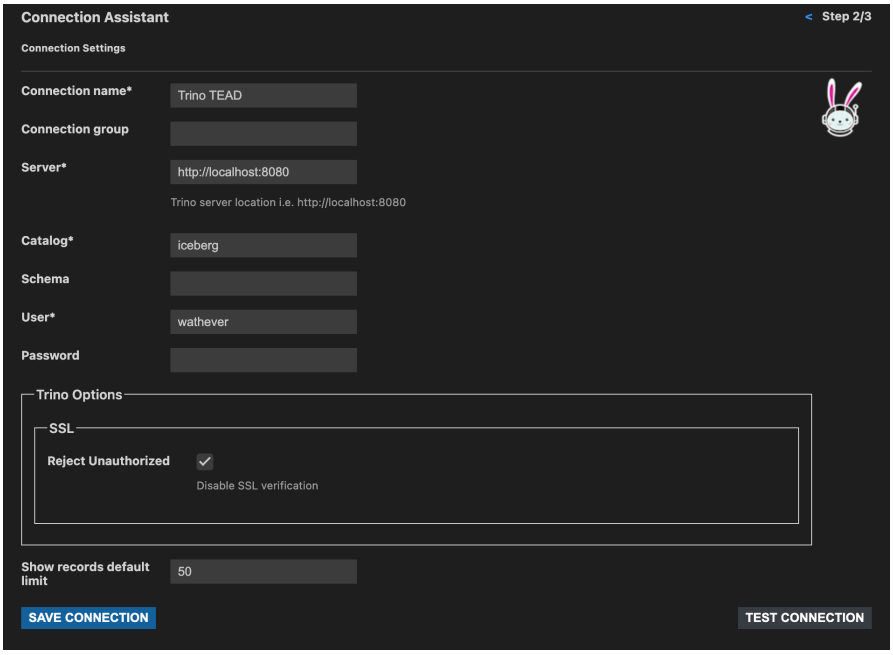
There are several ways of running Trino queries. The simplest one is to open the terminal in the Trino container and login to Trino there:

Alternatively, you can use any IDE that supports SQL + Trino driver. In VSCode, this can be configured as follows. Start by installing the following extensions.

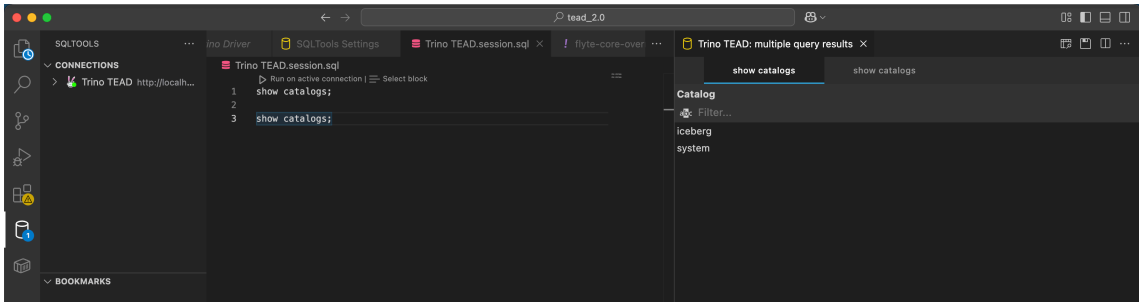




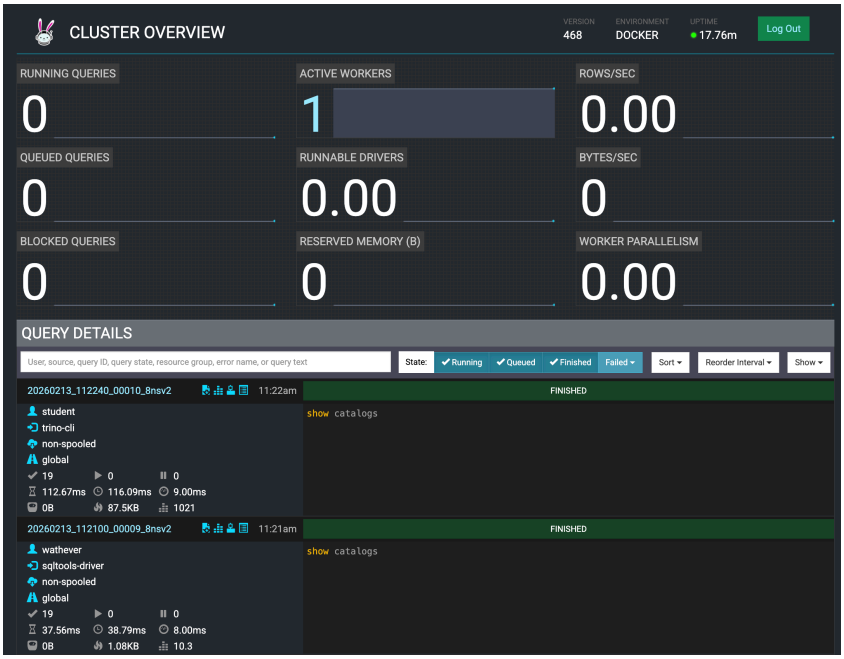
Then create a new connection using the Trino Driver, as shown below:



Finally, you can run sql files. If you have multiple queries, they will show in different tabs on the right:



In Trino's UI you can then check the status and statistics (e.g. run time, query plan) of each query:



1. Create a baseline table

In Trino, start by creating a new schema called tead in the iceberg catalog:

```
CREATE SCHEMA IF NOT EXISTS iceberg.tead
WITH (location = 's3a://warehouse/tead/');
```

Next, check if the schema was correctly created:

```
show schemas from iceberg;
```

Create a table with `age` and a derived `age_band` column, without partitioning. Trino supports CTAS (Create Table As Select), so this can be done with the following query:

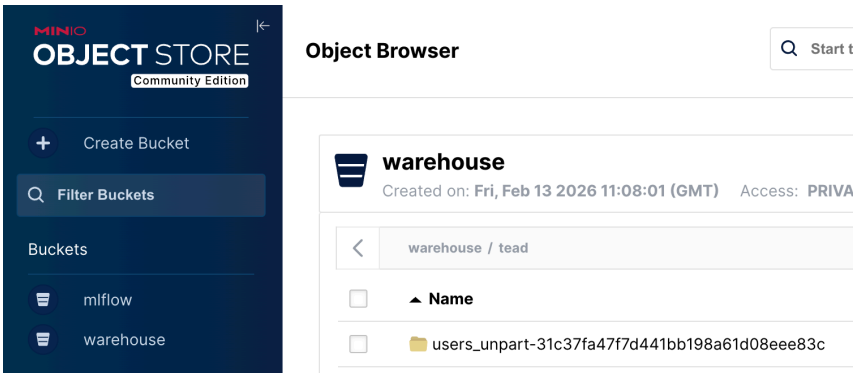
```
USE iceberg.tead;

CREATE TABLE iceberg.tead.users_unpart
WITH (format = 'PARQUET')
AS
WITH
  a AS (SELECT x FROM UNNEST(sequence(0, 999)) t(x)),      -- 1000 values
  b AS (SELECT y FROM UNNEST(sequence(1, 1000)) t(y)),     -- 1000 values
  base AS (
    SELECT x * 1000 + y AS user_id                        -- 1..1,000,000
    FROM a CROSS JOIN b
  )
SELECT
  user_id,
```

```
  date_add('day', -(user_id % 3650), current_date) AS signup_date,
  18 + (user_id % 63) AS age,
  CASE
    WHEN (18 + (user_id % 63)) BETWEEN 18 AND 24 THEN '18-24'
    WHEN (18 + (user_id % 63)) BETWEEN 25 AND 34 THEN '25-34'
    WHEN (18 + (user_id % 63)) BETWEEN 35 AND 44 THEN '35-44'
    WHEN (18 + (user_id % 63)) BETWEEN 45 AND 54 THEN '45-54'
    WHEN (18 + (user_id % 63)) BETWEEN 55 AND 64 THEN '55-64'
    ELSE '65+'
  END AS age_band,
  element_at(ARRAY['PT','ES','FR','DE','US','BR'], 1 + (user_id % 6))
AS country,
  element_at(ARRAY['free','basic','pro'], 1 + (user_id % 3)) AS plan
FROM base;
```

Check the data exists:[1]

```
SELECT count(*) FROM users_unpart;
SELECT age_band, count(*) FROM users_unpart GROUP BY 1 ORDER BY 1;
```

In MinIO you should also see a new folder in the warehouse bucket, with the respective folders for the data and meta-data, and the Parquet files:



2. Create a partitioned table

Create a new table with the same schema, but partitioned by `age_band`. This has a lower cardinality than if we used age, for instance, and probably offers enough granularity for the domain (although this would have to be checked and/or defined in the data contract). In any case, for the purpose of this tutorial, it will lead to fewer partitions.

Trino's Iceberg connector uses `partitioning = ARRAY[...]` for partition columns/transforms. We can thus create a partitioned table as:

```
CREATE TABLE users_by_age_band
```

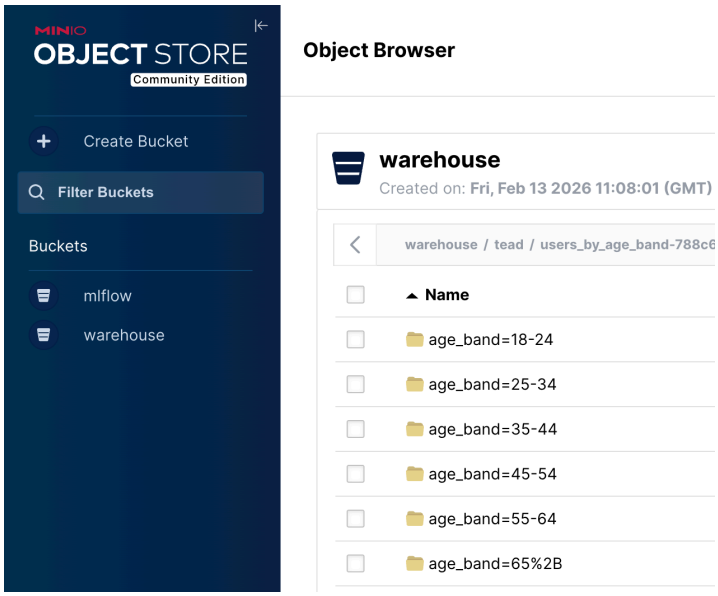
[1] https://trino.io/docs/current/connector/iceberg.html#partitioned-tables

```
WITH (
  format = 'PARQUET',
  partitioning = ARRAY['age_band']
)
AS
SELECT * FROM users_unpart;
```

Since Iceberg exposes metadata tables like `$partitions` and `$files`, we can observe the result of the partition:

```
SELECT * FROM "users_by_age_band$partitions" ORDER BY total_size DESC;

SELECT file_path, record_count, file_size_in_bytes
FROM "users_by_age_band$files"
ORDER BY file_size_in_bytes ASC
LIMIT 20;
```

The result of automatically partitioning the large file can also be observed in MinIO's UI:



3. Demonstrate pruning and why partition choice matters

Run the same query against both tables and compare how much data is scanned. In Trino UI (or query stats), compare processed input bytes/rows. The partitioned table should scan less because it can skip partitions (partition pruning), although results may not be significant due to the small size of the file(s).

```
SELECT count(*)
FROM users_by_age_band
WHERE age_band = '25-34';
```

```
SELECT count(*)
FROM users_unpart
WHERE age_band = '25-34';
```

Checking the performance of both queries on Trino's UI:

4. Inspect Iceberg metadata (like a Data Engineer!)

The goal is to connect the different levels of abstraction: "table" with "metadata" with "files in MinIO".

For snapshot history:

```
SELECT committed_at, snapshot_id, operation
FROM "users_by_age_band$snapshots"
ORDER BY committed_at DESC;
```

For table properties:

```
SELECT * FROM "users_by_age_band$properties";
SHOW CREATE TABLE users_by_age_band;
```

To list the partitions of the table:

```
SELECT * FROM "users_by_age_band$partitions";
```

To show how storage is abstracted (Iceberg exposes hidden metadata columns like `$partition` and `$path`):

```
SELECT
  partition,
```

```
  file_path,
  record_count,
  file_size_in_bytes
FROM "users_by_age_band$files"
ORDER BY file_size_in_bytes DESC;
```

Similar information can be obtained by exploring MinIO UI. In MinIO Console UI:

- Find the bucket/prefix where the table lives
- Identify:
  - Data/ objects (Parquet files)
  - Metadata/ objects (Iceberg metadata json/avro files)

Specifically, check the MinIO objects for `users_by_age_band` (data + metadata).

5. Create the "small files problem" (on purpose)

The goal is to experiment on how many small writes leads to many small files, which leads to overhead.

Create a fresh table for small-file demo:

```
CREATE TABLE users_smallfiles
WITH (
  format = 'PARQUET',
  partitioning = ARRAY['age_band']
)
AS
SELECT * FROM users_unpart WHERE 1=0;  -- empty table, same schema
```

Do multiple small inserts (repeat 5–8 times with different ranges):

```
INSERT INTO users_smallfiles
SELECT * FROM users_unpart
WHERE user_id BETWEEN 1 AND 20000;

INSERT INTO users_smallfiles
SELECT * FROM users_unpart
WHERE user_id BETWEEN 20001 AND 40000;

INSERT INTO users_smallfiles
SELECT * FROM users_unpart
WHERE user_id BETWEEN 40001 AND 60000;
```

Observe file explosion:

```
SELECT count(*) AS file_count FROM "users_smallfiles$files";
```

```
SELECT age_band, sum(file_count) AS files
FROM "users_smallfiles$partitions"
GROUP BY 1
ORDER BY files DESC;
```

6. Fix it with compaction (OPTIMIZE) and check results

The goal is to run Iceberg compaction to merge small files.

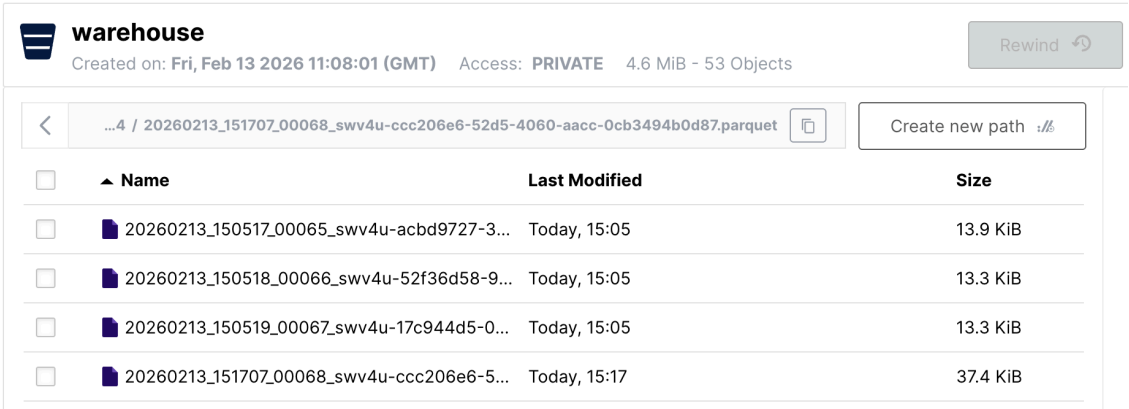
Trino supports ALTER TABLE … EXECUTE OPTIMIZE for Iceberg (compaction). Run compaction:

```
ALTER TABLE users_smallfiles
EXECUTE optimize(file_size_threshold => '128MB');
```

Check before/after

```
SELECT count(*) AS file_count FROM "users_smallfiles$files";

SELECT committed_at, snapshot_id, operation
FROM "users_smallfiles$snapshots"
ORDER BY committed_at DESC;
```

We're expected to observe fewer but larger files, as well as a new snapshot created. If we had 3 small files per age band, we will now have 3 (old small files) + 1 (new large file) per age band (as long as the new files are < 128MB). However, if we count the files, we will only get 6 (because Trino only looks at the most recent snapshot, although the files are all visible on MinIO):



7. Time Travel

The goal is to query the table of a previous snapshot, to show actions don't alter previous versions of the data. In Trino, time travel queries are supported via `FOR VERSION AS OF`...[2].

Pick an older `snapshot_id` from (we're casting snapshot ids to string as showing them as BigInts sometimes truncates them, rendering invalid IDs):

```
SELECT committed_at, CAST(snapshot_id AS VARCHAR) AS snapshot_id,
CAST(parent_id AS VARCHAR) AS parent_id, operation
FROM "users_smallfiles$snapshots"
ORDER BY committed_at DESC;
```

There are 5 snapshots. The initial one (parent_id = NULL), when the empty table was created, then three more append ones (one for each insert), and finally a replace one (resulting from the compaction operation).

committed_at	snapshot_id	parent_id	operation
Filter...	Filter...	Filter...	Filter...
2026-02-13 15:17:07.764 UTC	2949147897013742294	8794698337464491318	replace
2026-02-13 15:05:19.310 UTC	8794698337464491318	8262595124387733790	append
2026-02-13 15:05:18.646 UTC	8262595124387733790	4516086951492371671	append
2026-02-13 15:05:17.828 UTC	4516086951492371671	8316986775503889883	append
2026-02-13 15:02:48.284 UTC	8316986775503889883	NULL	append

Then query historical states vs. current state (for example):

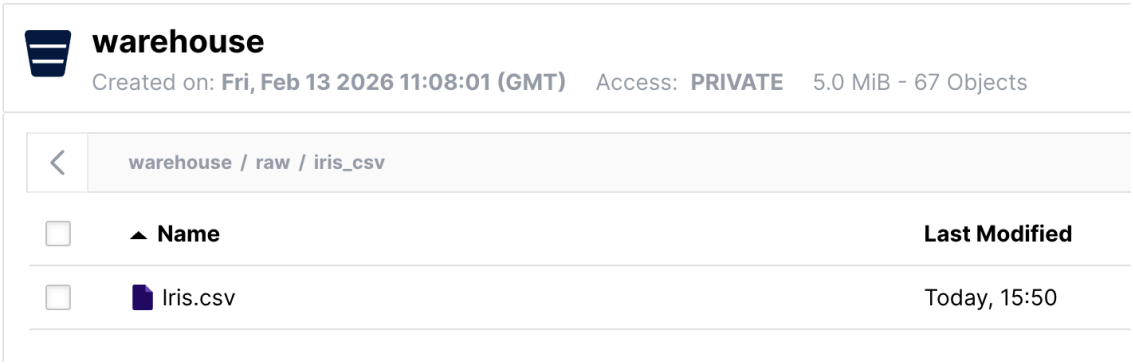
```
-- First snapshot
SELECT count(*)
FROM tead.users_smallfiles
FOR VERSION AS OF 8316986775503889883;

-- After the tree inserts
SELECT count(*)
FROM tead.users_smallfiles
FOR VERSION AS OF 8794698337464491318;

-- Latest one
SELECT count(*)
FROM tead.users_smallfiles;
```

[2] https://trino.io/docs/current/connector/iceberg.html#using-snapshots

8. Handling CSV files

Let us now ingest data from a .CSV file. The process involves several steps. Iceberg can't create a table directly from CSV, because Iceberg data files are stored in Parquet/ORC/Avro, not CSV. So we need to use a Hive connector, and then CTAS into Iceberg (Parquet).

Start by creating a new path on MinIO (`/warehouse/raw/iris_csv/`) and uploading the classic iris.csv dataset.



Next, create an external table in Hive to which import CSV (this works mostly as a placeholder/registration, it does not really hold data, so it does not show up in MinIO). Trino's Hive connector supports `format='CSV'` plus CSV-specific properties like `csv_separator`, `csv_quote`, `csv_escape`, and header skipping via `skip_header_line_count`.

```
CREATE SCHEMA IF NOT EXISTS hive.tead
WITH (location = 's3a://warehouse/tead/');

CREATE TABLE hive.tead.iris_csv (
  Id               varchar,
  SepalLengthCm    varchar,
  SepalWidthCm     varchar,
  PetalLengthCm    varchar,
  PetalWidthCm     varchar,
  Species          varchar
)
WITH (
  format = 'CSV',
  external_location = 's3a://warehouse/raw/iris_csv/',
  skip_header_line_count = 1,
  csv_separator = ',',
  csv_quote = '"',
  csv_escape = '"'
);
```

Read the data from Hive to see if it is ok:

```
SELECT *
FROM hive.tead.iris_csv
LIMIT 20;
```

Next, create the Iceberg table (stored as Parquet) by reading from (through) the Hive table, with CTAS:

```
CREATE TABLE iceberg.tead.iris
WITH (format = 'PARQUET')
AS
SELECT
  try_cast(id as integer) AS id,
  try_cast(SepalLengthCm as double) AS SepalLengthCm,
  try_cast(SepalWidthCm as double) AS SepalWidthCm,
  try_cast(PetalLengthCm as double) AS PetalLengthCm,
  try_cast(PetalWidthCm as double) AS PetalWidthCm,
  Species
FROM hive.tead.iris_csv
WHERE try_cast(id as integer) IS NOT NULL;
```

And finally check the data in Parquet:

```
SELECT * FROM tead.iris
LIMIT 20
```